

RELATÓRIO DE AUDITORIA TÉCNICA, FORENSE E DE CONFORMIDADE

Avaliação de Segurança da Informação, LGPD, Stack Tecnológica e Arquitetura no projeto *OrganizaAí*

Prof. Dr. Fabiano Menegidio
Universidade de Mogi das Cruzes - UMC

25 de maio de 2026

Resumo

O presente relatório consubstancia uma auditoria forense e arquitetural detalhada da API *OrganizaAí*, um sistema de gestão de eventos. A avaliação foi conduzida sob a ótica da norma ISO/IEC 27001 para gestão de segurança da informação e as diretrizes da Lei Geral de Proteção de Dados (LGPD). Através da análise estática da stack tecnológica (Java 21, Spring Boot 3.x), das lógicas de controle de acesso (JWT, 2FA/TOTP) e do padrão arquitetural, este documento mapeia a robustez do ecossistema, o débito técnico atrelado a ferramentas de IA e fornece um plano de remediação rigoroso para homologação em ambiente produtivo.

1 Dados de Identificação, Stack Tecnológica e Contexto Arquitetural

1.1 Identificação do Projeto

Nome do Projeto: OrganizaAí - Gestão de Eventos

Escopo: API RESTful para centralização e automação do ciclo de vida de eventos sociais, votação de datas e divisão financeira de insumos.

Módulos Críticos: Módulo de Autenticação/2FA, Gestão de Sessões via JWT, Processamento de Votos e Cálculo de Insumos.

1.2 Mapeamento Detalhado da Stack

A análise das dependências (`pom.xml`) e artefatos de configuração revela uma stack robusta e moderna, caracterizada pelos seguintes componentes:

- **Linguagem e Core:** Java 21 (LTS) operando sob o framework Spring Boot 3.5.13.
- **Segurança e Criptografia:** Spring Boot Starter Security, `jjwt` (0.12.5) para geração de tokens, `googleauth` (1.5.0) para TOTP e `zxing` para geração de QR Codes de pareamento físico.
- **Defesa Cibernética:** `bucket4j-core` (8.10.1) implementando o algoritmo *Token Bucket* para *Rate Limiting* e mitigação de DoS/Brute Force.
- **Persistência e Banco de Dados:** Spring Data JPA gerenciando entidades no MySQL (produção) e H2 (runtime/testes).
- **Comunicação e Observabilidade:** Springdoc OpenAPI (Swagger) para documentação e Spring Boot Mail/Thymeleaf para mensageria assíncrona.

1.3 Análise de Arquitetura de Software

O sistema emprega um padrão arquitetural de **Monólito RESTful em Camadas** (Controller, Service, Repository, Security/Config).

- **Fluxo de Dados (Data Flow):** Requisições externas trafegam via HTTP/TLS, sendo interceptadas por filtros de segurança Stateless configurados no Spring Security. Após verificação de Blacklist e assinatura HMAC-SHA384 do JWT, o *payload* atinge a camada Controller.
- **Isolamento e Fronteiras:** O acoplamento lógico está centrado no domínio do Evento. Há uma dependência forte entre o evento, participantes e insumos, característica de monólitos.
- **Pontos Críticos:** Sendo *Stateless*, a escalabilidade horizontal é viável. Contudo, a estratégia de *Logout Blacklist* em banco de dados relacional cria um gargalo síncrono de I/O de leitura para cada requisição autenticada, sendo o principal ponto único de falha de performance sob alta carga.

1.4 Enquadramento Regulatório (LGPD)

O sistema processa e persiste as seguintes tipologias de dados:

- **Dados Comuns (Art. 5º, I):** Nome, e-mail e identificadores transacionais.
- **Dados Potencialmente Sensíveis (Art. 5º, II):** A variável comportamental consome *Alcool* do modelo Participante. O rastreamento de hábitos de consumo de substâncias, quando atrelado a um indivíduo identificável, tangencia a esfera de dados sensíveis (saúde/hábitos de vida), exigindo tratamento rigoroso.
- **Base Legal:** A coleta é fundamentada no **Consentimento** (Art. 7º, I) e **Execução de Contrato/Serviço** (Art. 7º, V).

2 Avaliação de AI-Assisted Development (Engenharia de Software)

Através de heurísticas de auditoria estática e análise dos arquivos estruturais (*pom.xml*, justificativas), nota-se um desenvolvimento **Híbrido**, com forte direcionamento e otimização por Inteligência Artificial (*Vibecoding / AI-Driven*).

Evidências de Débito Técnico atreladas ao uso de LLMs:

1. **Sobreengenharia de Filtros (Redundância):** O uso de JWT com Blacklist armazenada em banco relacional quebra o princípio primário da arquitetura *Stateless*. Um LLM frequentemente sugere a persistência de JWTs revogados em SQL ao invés de utilizar datastores em memória (ex: Redis) ou encurtar o TLL (*Time-to-Live*) do token.
2. **Tratamento Genérico de Exceções:** A documentação e estrutura indicam a possibilidade de Controladores contendo lógicas onde exceções genéricas (*Exception*) podem mascarar vetores de erro granulares (ex: *DataIntegrityViolationException*), padrão comum em abstrações geradas por IA.
3. **Estrutura de Commits e Comentários:** O detalhamento do arquivo de justificativas apresenta padronização textual característica de sínteses automatizadas, embora tecnicamente preciso.

Score de Qualidade de Código / Uso de IA: 82/100

Justificativa: A adoção de *Bucket4j* e *GoogleAuth* demonstra refinamento arquitetural sênior, porém os gargalos na invalidação de JWT via DB reduzem a eficiência escalar.

3 Diagnóstico Baseado na Tríade SIA/CIDA

- **Confidencialidade (Score: 95/100):** Uso ostensivo do BCrypt (Work Factor = 12) com salt dinâmico, anulando ataques de *Rainbow Tables*. Variáveis de ambiente isoladas em `.env`.
- **Integridade (Score: 88/100):** O JWT é assinado via HMAC-SHA384, prevenindo adulteração de *payloads*. A dependência `spring-boot-starter-validation` atesta proteção contra injeções de parâmetros em DTOs.
- **Disponibilidade (Score: 78/100):** Uso do Bucket4j protege contra preenchimento automatizado de credenciais. Contudo, o I/O síncrono no banco relacional (MySQL) para verificação de *Blacklist* afeta a latência e a disponibilidade sob tráfego massivo.
- **Autenticidade (Score: 98/100):** Excelência alcançada pela obrigatoriedade de 2FA/TOTP acoplado ao *Google Authenticator* e senhas em hash lento. Imunidade a ataques de *SIM Swapping* comprovada.

4 Modelagem de Ameaças e Resiliência a Ataques

A topologia REST expõe a superfície do *OrganizaAí* aos seguintes vetores de risco, para os quais o sistema possui contramedidas mistas:

- **Credential Stuffing e Brute Force:** Risco **Mitigado**. O atraso exponencial mitigado por Bucket4j (bloqueio após 5 falhas por 15 min) impede a extração algorítmica de senhas.
- **Insecure Deserialization / JWT Spoofing:** Risco **Baixo**. A adoção de HMAC-SHA384 e o armazenamento em segredo estrito (JWT_SECRET via ENV) repelem o ataque de assinatura "None" em algoritmos JWT.
- **Exaustão de Recursos de Infraestrutura (DoS Lógico):** Risco **Médio/Alto**. O cálculo complexo do método `calcularCustoTotal()` iterando sobre extensas listas do JPA sem paginação explícita pode causar exaustão de CPU (Memory Leak) se houver injeção de milhares de participantes num único evento.
- **Supply Chain Attacks:** Risco **Médio**. Dependências transitivas no `pom.xml` requerem auditoria contínua via ferramentas de SCA (Software Composition Analysis) para prevenir a execução de código arbitrário.

5 Auditoria Baseada no Checklist de Requisitos

A tabela abaixo correlaciona os requisitos funcionais e de conformidade de segurança (itens 1 a 8), auferidos mediante a varredura da arquitetura e das justificativas providas do *OrganizaAí*.

Nº	Requisito	Status	Justificativa e Evidência Técnica
1. Autenticação e Gestão de Credenciais			
1.1	Hash criptográfico seguro	CONF.	Utilizado algoritmo BCrypt padrão da indústria.
1.2	Parâmetros de custo ajustados	CONF.	Work Factor (Strength) = 12 aplicado.
1.3	Salt criptográfico único	CONF.	BCrypt implementado garante geração de <i>salt</i> dinâmico no payload.
1.5	2FA Implementado	CONF.	Uso do TOTP via googleauth RFC 6238.
1.9	Sessões com expiração	CONF.	JWT implementa claims (exp) curtos.
1.10	Invalidação de sessão	CONF.	Implementada tabela de Blacklist transaccional.
1.11	Proteção Brute Force	CONF.	Bloqueio por 15 min via Bucket4j após 5 falhas.
2. Recuperação de Senha			
2.1	Funcionalidade de recuperação	CONF.	Presença de <code>spring-boot-starter-mail</code> e credenciais do Mailtrap no <code>.env</code> .
2.4	Token invalidado após uso	N.E.	Falta evidência estrita da inativação transaccional na base de dados após o <i>reset</i> .
3. Criptografia e Comunicação Segura			
3.1	Comunicação HTTPS/TLS	N.E.	Nenhuma configuração de <i>Keystore</i> em <i>application.properties</i> foi exibida na documentação inicial.
3.4	Dados em repouso cifrados	N.C.	Salvo as senhas, dados de participantes (email) não apresentam cifragem em nível de TDE ou coluna.
4. Conformidade com a LGPD			
4.2	Associação à finalidade	CONF.	A flag <code>consumeAlcool</code> serve exclusivamente ao método de partição financeira.
4.10	Exclusão de dados (Direito)	N.E.	Ausência de rota confirmada de apagamento lógico/físico do usuário (<i>Right to be Forgotten</i>).
5 e 6. Auditoria, Logs e Documentação			
5.1	Logs de Autenticação	CONF.	Tratamento assíncrono via filtros e <i>SecurityContext</i> .
6.2	Diagrama de Arquitetura	CONF.	O arquivo <code>readme.md</code> contém diagrama funcional em sintaxe <i>Mermaid</i> .
6.8	Risco x Contramedida	CONF.	Documento de justificativas detalha perfeitamente as mitigações (ex: TOTP vs SIM Swapping).
7 e 8. Qualidade Científica do Escopo			

Continua na próxima página...

Nº	Requisito	Status	Justificativa e Evidência Técnica
7.4	Mecanismos de segurança	CONF.	Explicitamente descritos no documento justificativas.md.
8.2	Arquitetura visual	CONF.	<i>Mermaid</i> gerencia relações (1:N) adequadamente.

Legenda de Status: [CONF.] Conforme / [N.C.] Não Conforme / [N.E.] Não Evidenciado.

6 Matriz de Risco, Plano de Ação e Remediação Arquitetural

Vulnerabilidade / Risco	Pilar	Severidade	Impacto Técnico	Ação / Remediação
1. Exaustão de Banco de Dados via Blacklist de JWT.	Disp.	Média	Acúmulo de requisições de I/O por <i>request</i> pode gerar <i>Timeout</i> .	Migrar a validação da <i>Blacklist</i> para um datastore em memória veloz, como o Redis .
2. Mascaramento de Exceções Lógicas (Padrão LLM).	Integ.	Média	Vasamento do <i>StackTrace</i> ao usuário final facilitando reconhecimento.	Implementar <code>@ControllerAdvice</code> global garantindo retornos estritamente conformes ao padrão RFC 7807 (Problem Details).
3. Superexposição em <i>Endpoints</i> Não Paginados.	Disp.	Alta	Exaustão de memória da JVM ao retornar todos os participantes.	Adotar <i>Pageable</i> e limites rígidos no escopo do <i>JpaRepository</i> .
4. Perfilamento Não Anonimizado (Flag de Álcool).	Conf.	Alta	Vazamento de características de hábitos do titular; quebra LGPD.	Cifrar a flag ou mascarar <i>payloads</i> na DTO de retorno, impedindo extração em massa via <i>Scraping</i> .

6.1 Implementação Prática de Remediação (*Secure Coding*)

A fim de suprir a mitigação **Risco 2 (Mascaramento de Exceções)** originada possivelmente pela adoção de blocos autogerados por LLM, apresentamos a implementação de um manipulador global rigoroso em Java (Spring Boot) evitando *Information Disclosure*:

```

1 package com.organizaai.config.exception;
2
3 import org.springframework.http.HttpStatus;
4 import org.springframework.http.ProblemDetail;
5 import org.springframework.web.bind.annotation.ControllerAdvice;
6 import org.springframework.web.bind.annotation.ExceptionHandler;
7
8 import java.net.URI;
9 import java.time.Instant;
10
11 @ControllerAdvice
12 public class GlobalSecurityExceptionHandler {
13
14     // Bloqueia a exposicao da StackTrace para excecoes genericas
15     @ExceptionHandler(Exception.class)
16     public ProblemDetail handleGenericException(Exception ex) {
17         ProblemDetail problemDetail = ProblemDetail.forStatusAndDetail(
18             HttpStatus.INTERNAL_SERVER_ERROR,
19             "Ocorreu um erro interno de processamento. A acao foi logada."
20         );
21         problemDetail.setType(URI.create("https://api.organizaai.com/errors/internal"));
22         problemDetail.setTitle("Internal Server Error");
23     }
24 }

```

```
23     problemDetail.setProperty("timestamp", Instant.now());
24
25     // Log seguro na infraestrutura sem devolver ao client
26     // logger.error("Falha Critica: ", ex);
27
28     return problemDetail;
29 }
30 }
```

Listing 1: Implementação Global Segura de Manipulação de Erros (RFC 7807)

Para a mitigação da arquitetura e proteção contra ataques DoS e Brute force abordada nas documentações fornecidas, segue um modelo defensivo de integração do *Bucket4j* atuando como *Middleware Filter*:

```
1 package com.organizaai.config.security;
2
3 import io.github.bucket4j.Bucket;
4 import jakarta.servlet.*;
5 import jakarta.servlet.http.HttpServletResponse;
6 import java.io.IOException;
7
8 public class RateLimitingFilter implements Filter {
9
10     private final Bucket bucket;
11
12     public RateLimitingFilter(Bucket bucket) {
13         this.bucket = bucket;
14     }
15
16     @Override
17     public void doFilter(ServletRequest request, ServletResponse response,
18         FilterChain chain)
19         throws IOException, ServletException {
20
21         if (bucket.tryConsume(1)) {
22             chain.doFilter(request, response); // Permite o fluxo
23         } else {
24             HttpServletResponse httpResponse = (HttpServletResponse) response;
25             httpResponse.setStatus(429); // HTTP 429 Too Many Requests
26             httpResponse.getWriter().write("Limite de tentativas excedido.
27 Bloqueio ativo.");
28         }
29     }
30 }
```

Listing 2: Interceptor Defensivo contra Brute Force (Bucket4j)

7 Conclusão do Parecer de Auditoria

O projeto *OrganizaAí* apresenta uma arquitetura sólida calcada nas premissas de *Security by Design*. A incorporação de criptografia pesada (BCrypt, fator 12) associada ao bloqueio de requisições temporais via *Bucket4j* e adoção de MFA Físico (Google Auth / TOTP) demonstram maturidade excepcional na proteção de ativos do usuário, atingindo ampla conformidade com normativas e a LGPD.

Contudo, como arquiteto de soluções, denota-se que o pilar da Disponibilidade carece de modernização. A estrutura da invalidação Stateless (*Blacklist*) sobre banco relacional fere o princípio da ausência de estado, requerendo transição para *caches* em memória a fim de garantir estabilidade sob picos de transações. Sanados esses pontos apontados e validada a criptografia em repouso dos dados de consumo e perfil, o sistema encontra-se tecnicamente viável e referendado para homologação e deploy em ambiente de produção contínua.

Referências

- [1] ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. *ABNT NBR ISO/IEC 27001:2022 - Segurança da informação, segurança cibernética e proteção de privacidade — Sistemas de gestão da segurança da informação — Requisitos*. Rio de Janeiro: ABNT, 2022.
- [2] BRASIL. *Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD)*. Diário Oficial da União, Brasília, DF, 15 ago. 2018.
- [3] OWASP Foundation. *OWASP Top 10:2021 - The Ten Most Critical Web Application Security Risks*. Disponível em: <https://owasp.org/Top10/>. Acesso em: Mai. 2026.
- [4] NOTTINGHAM, M.; WILDE, E. *RFC 7807: Problem Details for HTTP APIs*. IETF, Março 2016.